

Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

Walaa H. Elashmawi
Professor of AI

 walaa.hassan@miuegypt.edu.eg

Supervised Machine Learning Models



Classification in ML

- **Classification** is a supervised machine learning technique that aims at categorizing a set of data into different classes. In other words, if we think of a dataset as a set of data instances, and each data instance as a set of features, then Classification is the process of predicting the particular class that that individual data instance might belong to, based on its features.
- **The Classification algorithm's task** is approximating the mapping function so we can map the input variable of "x" to the discrete output variable of "y", which can be labels or categories.



Difference between Classification & Regression ??

Difference Between Classification & Regression

Classification

Classification predicts discrete class labels.

Regression

Regression helps predict a continuous quantity.

Some Overlaps

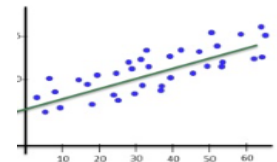
A classification algorithm can predict a continuous value if it is in the form of a class label probability.

Ex: analyze whether an email is a spam or not spam. "checks the keywords in an email and the sender's address is to find out the probability of the email being spam".



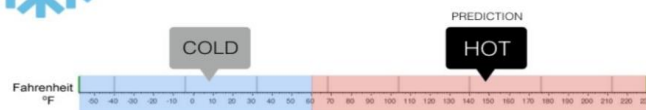
A regression algorithm can predict a discrete value which is in the form of an integer quantity.

Ex: predict the height of any student based on their weight, gender, diet, or subject major. "height is a continuous quantity. There is an infinite number of possible values for a person's height".



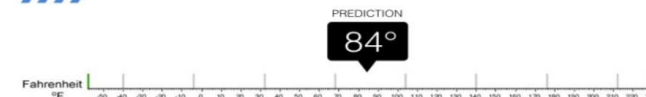
Classification

Will it be Cold or Hot tomorrow?



Regression

What is the temperature going to be tomorrow?



Types of classification problems

- **Binary Classification:**

Two classes (positive/negative).

Examples: disease detection, spam detector, fraud detection.

- **Multi-class Classification:**

More than two mutually exclusive classes and each data sample is assigned to one and only one class.

Examples: sentiment analysis, image classification, document categorization.

- **Multi-label Classification:**

Each sample can belong to multiple classes simultaneously.

Examples: image tagging, multi-topic text categorization, multi-condition medical diagnosis.

Binary Classification



- Spam
- Not spam

Multiclass Classification



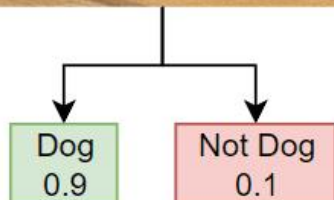
- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label Classification

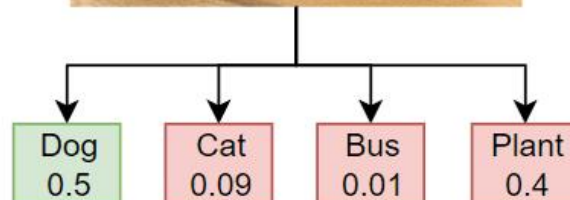


- Dog
- Cat
- Horse
- Fish
- Bird
- ...

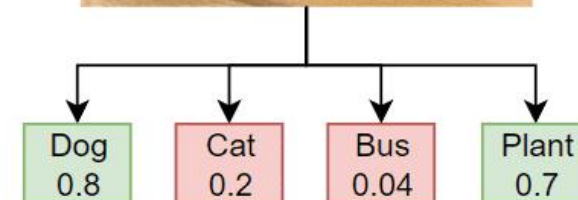
Binary Classification



Multiclass Classification

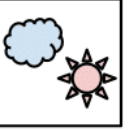


Multilabel Classification



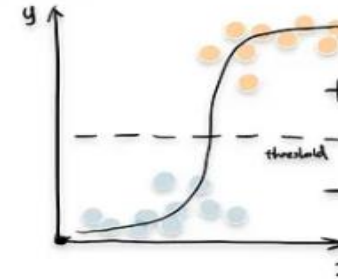
Multi-Class

Multi-Label

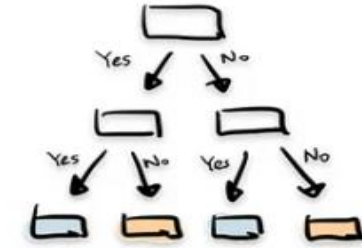
C = 3		
Samples		
		
		
		
Labels (t)		
	[0 0 1]	[1 0 0]
		[0 1 0]
Samples		
		
Labels (t)		
	[1 0 1]	[0 1 0]
		[1 1 1]

Classification Techniques

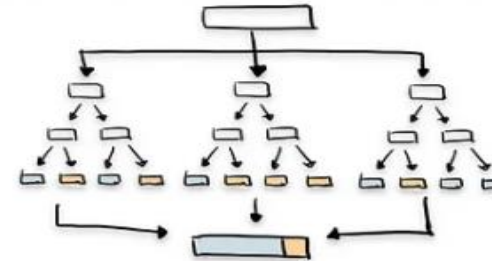
Logistic Regression



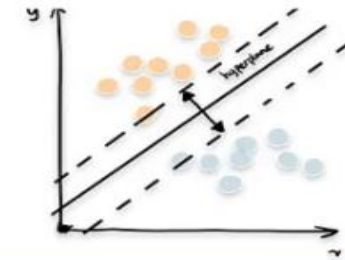
Decision Tree



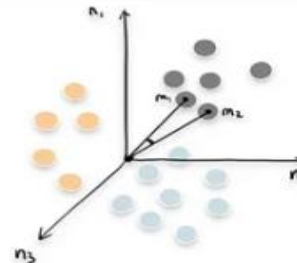
Random Forest



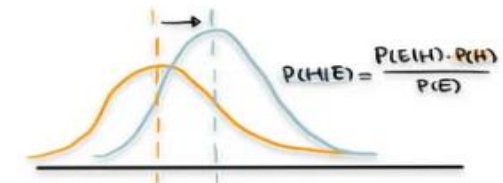
Support Vector Machine



K Nearest Neighbour

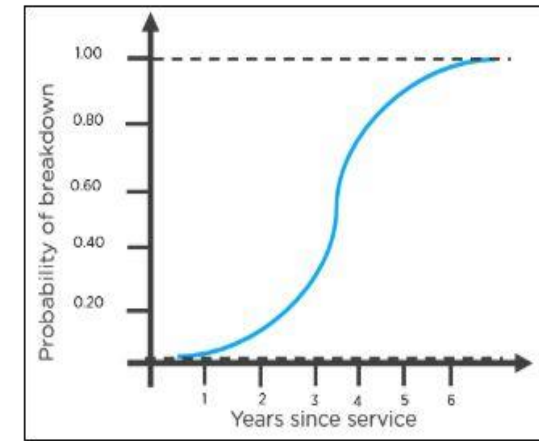


Naive Bayes



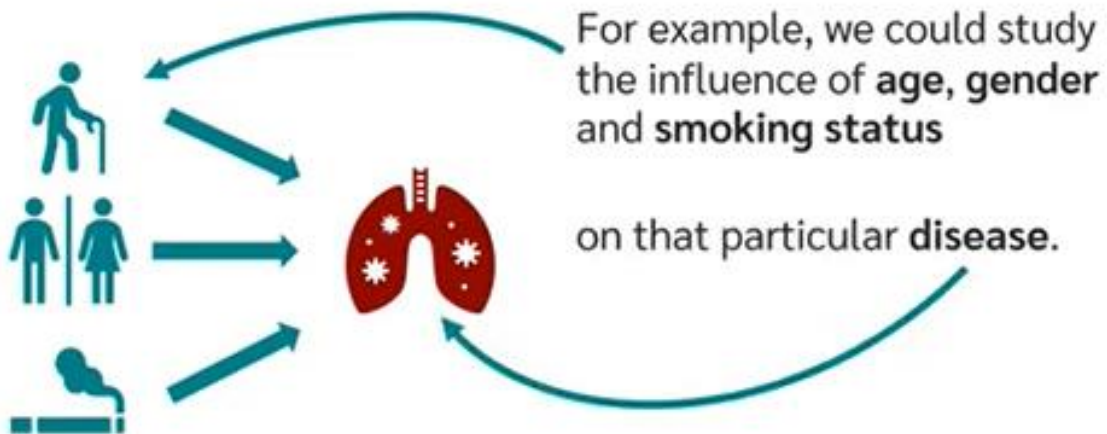
Logistic Regression

What is Logistic Regression?



- **Logistic regression** is a statistical method that is used for building machine learning models for binary classification (i.e., it's designed to predict the probability that an input instance belongs to one or two possible classes).
- Despite its name, logistic regression is used for classification rather than regression tasks. It is derived from the concept of the logistic function that it uses. The logistic function is also known as the sigmoid function. The value of this logistic function lies between zero and one.

- With the help of logistic regression, we can determine what has an influence on whether a certain disease is **present** or **not**.



In this case
0 stands for **not diseased** and **1** for **diseased**.



and the **probability** for the occurrence of the **characteristic 1** (=characteristic present) is estimated.

Our dataset might look like this...

Here we have the
independent variables

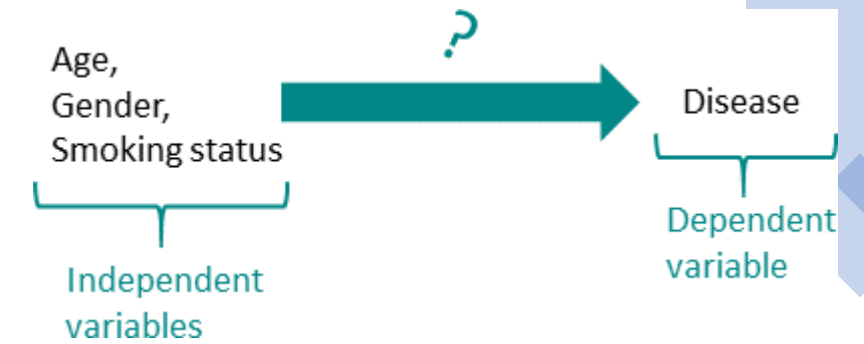
and here the **dependent variable** with 0 and 1.

Age	Gender	Smoker status	Disease
22	female	Non-smoker	1
25	female	Smoker	1
18	male	Smoker	0
45	male	Non-smoker	0
12	female	Smoker	0
43	male	Smoker	1
23	male	Smoker	0
33	male	Smoker	1
...	...	...	...

We could now investigate what influence the **independent variables** have on the disease.



Age	Gender	Smoker status	Disease
22	female	Non-smoker	1
25	female	Smoker	1
18	male	Smoker	0
45	male	Non-smoker	0
12	female	Smoker	0
43	male	Smoker	1
23	male	Smoker	0
33	male	Smoker	1
...	...	...	...



If there is an influence, then we can **predict** how **likely** a person is to have a certain disease.

In this case, why we need a **Logistic regression** instead of Linear regression?

This is what have we done before.

In **linear regression**, this is our **regression equation**.

$$\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$$

We have the
dependent variable

the
independent variables

a is the intercept
 b_i is the coefficient
 x_i is the features



The goal is to find the optimal
values of b s and a

The diagram shows the regression equation $\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$ with components highlighted in boxes. A red arrow points from the text 'We have the dependent variable' to the $\hat{y}$ box. Two blue arrows point from the text 'the independent variables' to the x_1 and x_2 boxes. Four blue arrows point from the text 'and the regression coefficients.' to the b_1 , b_2 , b_k , and a boxes.

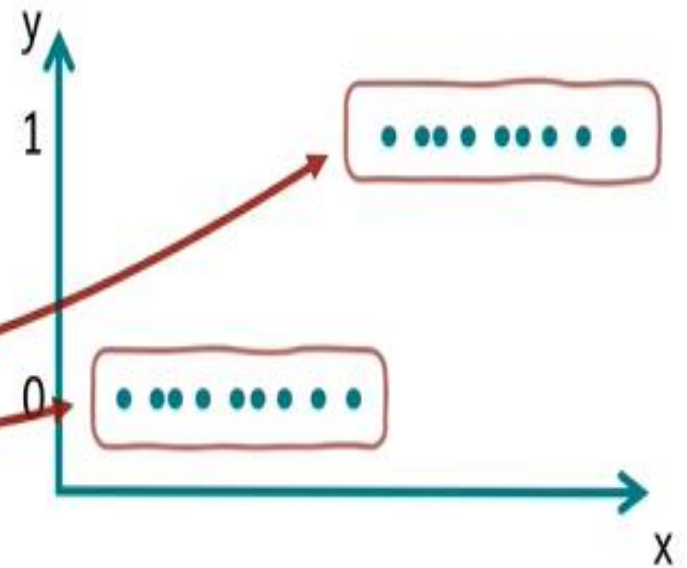
and the **regression coefficients**.

$$\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$$

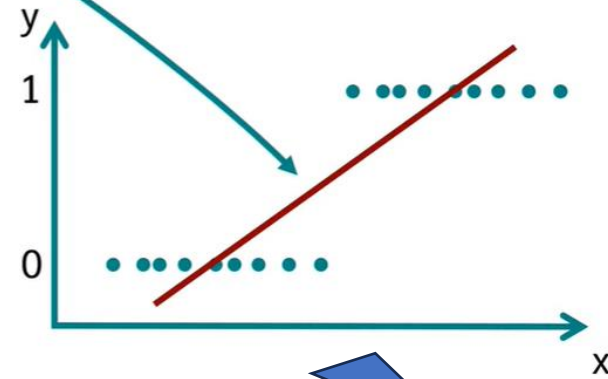
However, we now have a **dependent variable** that is either **0** or **1**.



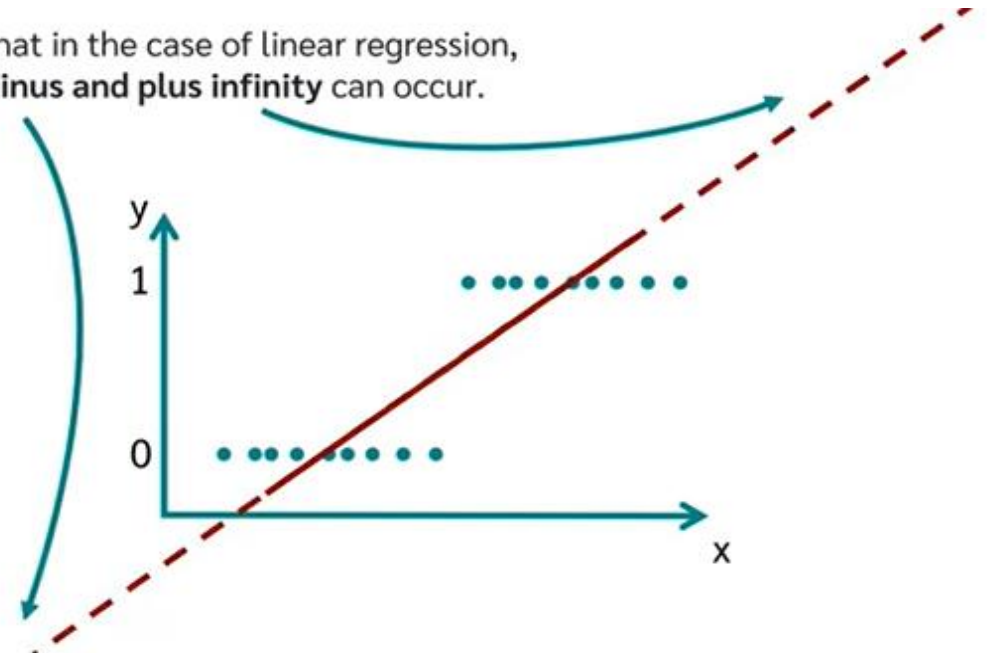
No matter which value we have for the **independent variables**, only **0** or **1** results.



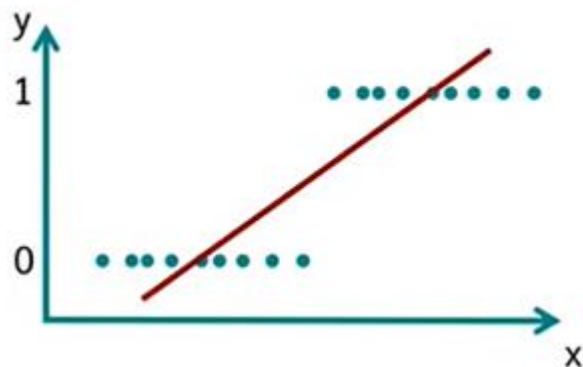
A **linear regression** would now simply put a **straight line** through the **points**.



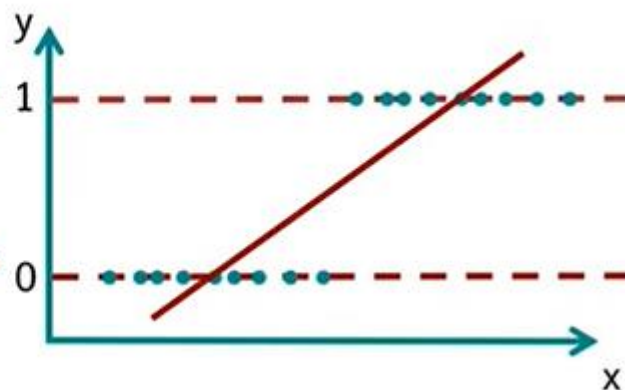
We can now see, that in the case of linear regression, values between **minus** and **plus infinity** can occur.



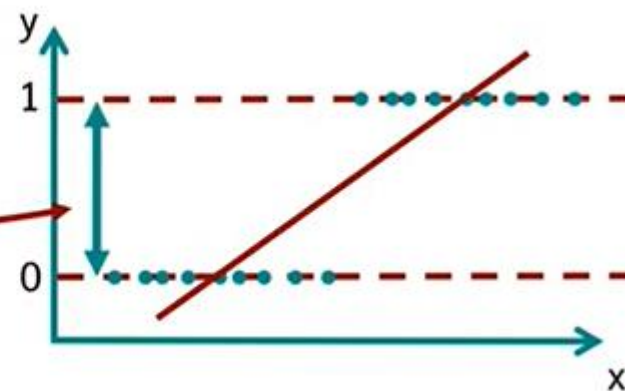
However, the goal of logistic regression is to estimate the **probability** of occurrence.



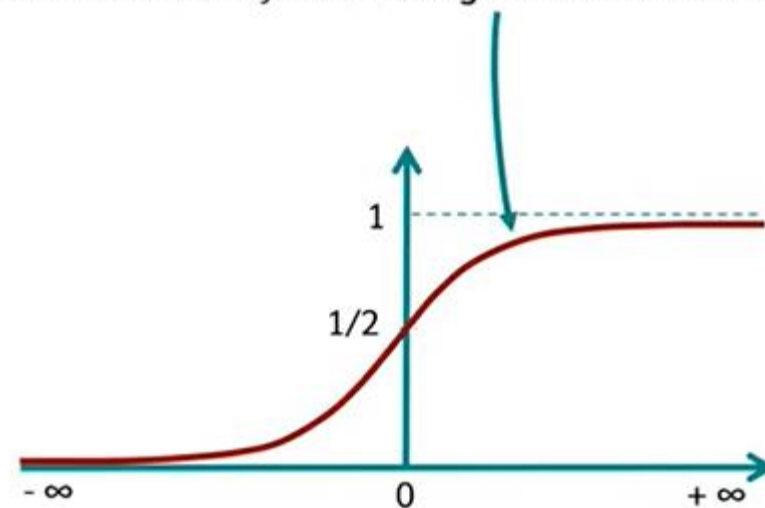
The value range for the prediction should therefore be between **0** and **1**.



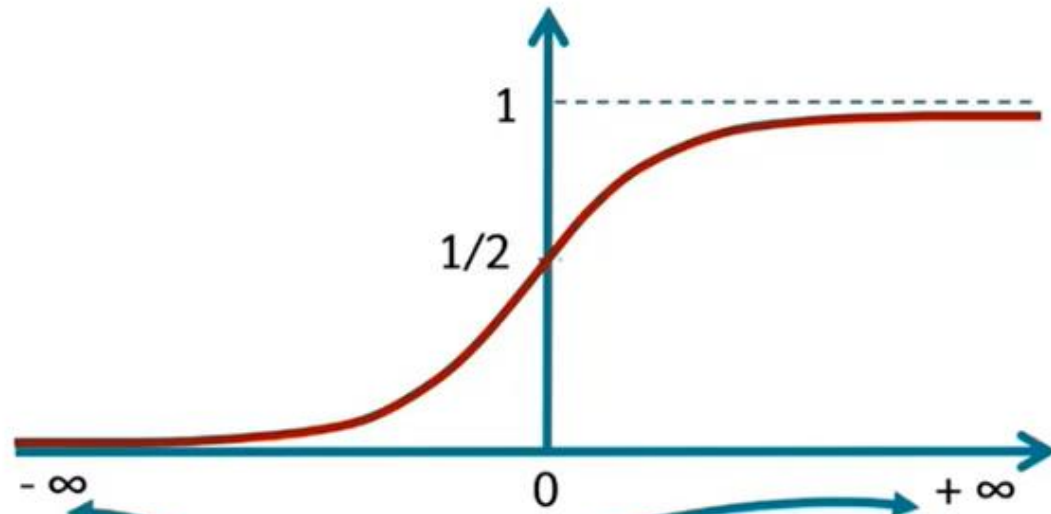
So we need a function that only takes values between **0** and **1**!



And that is exactly what the **logistic function** does.



No matter where we are on the x-axis,



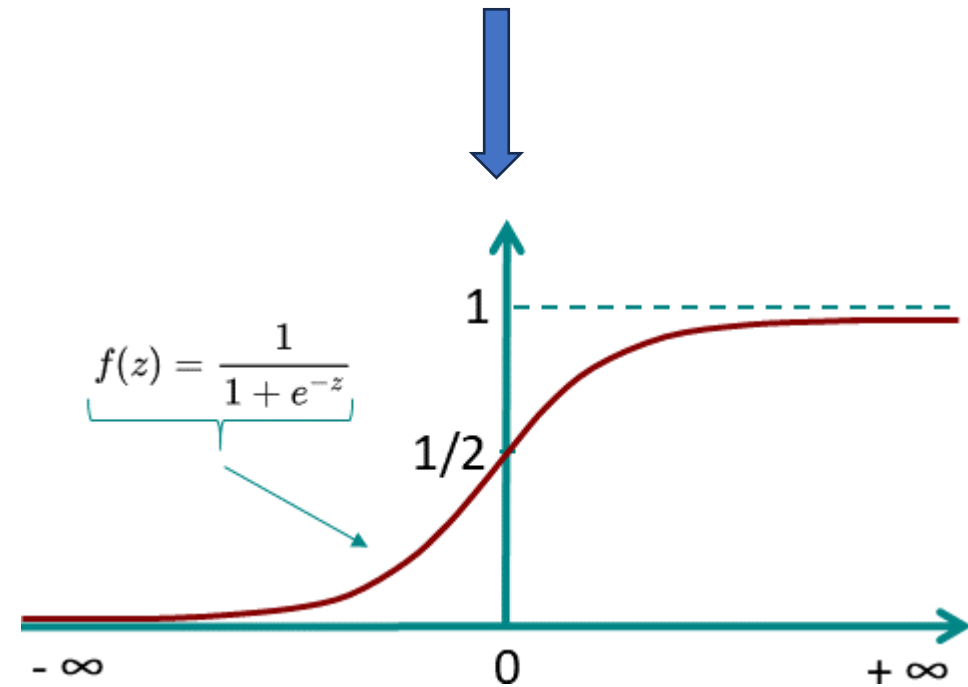
between **minus** and **plus infinity**
only values between **0** and **1** result.

And that is
exactly what
we want!

The **equation** for the **logistic function** looks like this:

$$f(z) = \frac{1}{1 + e^{-z}}$$

We have 1 divided by 1 plus e^{-z} .



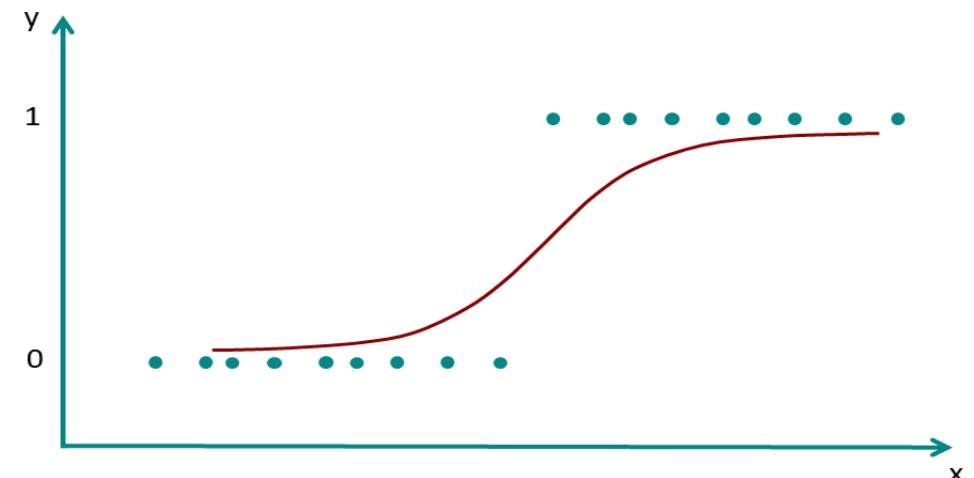
Logistic regression

- So, the logistic function is perfect to describe the **probability** $P(y=1)$. If the logistic function is now applied to the upper regression equation the result is:

$$\hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$$

$$f(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(b_1 \cdot x_1 + \dots + b_k \cdot x_k + a)}}$$

- This now ensures that no matter in which range the x values are located, only values between 0 and 1 will come out. The new graph now looks like this:



- Logistic regression used for predicting the categorical dependent variable (output) using a given set of independent variables. Therefore, the outcome must be a categorical or discrete value. It can be either Yes/No, 0/1, True/False,.. But instead of giving exact value as 0 and 1, **it gives a probabilistic values which lie between 0 and 1.**
- Logistic regression is much like linear regression except how it can be used. Linear regression used for solving regression problems, whereas logistic regression is used for solving the classification problems.

- The probability that for given values of the independent variable, the dichotomous dependent variable y is 0 or 1 is given by:

$$P(y = 1|x_1, \dots, x_n) = \frac{1}{1 + e^{-(b_1 \cdot x_1 + \dots + b_k \cdot x_k + a)}}$$

$$P(y = 0|x_1, \dots, x_n) = 1 - \frac{1}{1 + e^{-(b_1 \cdot x_1 + \dots + b_k \cdot x_k + a)}}$$

- To calculate the probability of a person being sick or not using the logistic regression for the example above.
- The model parameters b_1, b_2, b_3 and a must first be determined.

In our example,
the **probability** of having a **certain disease**

$$P(is\ diseased) = \frac{1}{1 + e^{-(b_1 \cdot Age + b_2 \cdot Male + b_3 \cdot Smoker + a)}}$$

is a function of **age**, **gender** and **smoking status**.

$$P(is\ diseased) = \frac{1}{1 + e^{-(b_1 \cdot Age + b_2 \cdot Male + b_3 \cdot Smoker + a)}}$$

Now we need to determine the **coefficients** so that our model best represents the given data.



To determine the model parameters for the **logistic regression equation**, the **Maximum Likelihood Method** is applied. The maximum likelihood method is one of several methods used in statistics to estimate the parameters of a mathematical model by maximizing the likelihood function.

- The **Maximum Likelihood Method (MLM)** is widely employed in various fields including ML and econometrics.
- The primary idea behind MLM is to find the parameter values that make the observed data most likely to have been generated under the assumed model.

In other words,

In Maximum Likelihood Estimation, a probability distribution for the target variable (class label) is assumed and then a likelihood function is defined that calculates the probability of observing the outcome given the input data and the model. This function can then be **optimized to find the set of parameters that results in the largest sum likelihood over the training dataset.**

Maximum Likelihood Estimation (MLE) is equivalent to minimizing the cross-entropy (log-loss) function *for logistic regression*.

Proof

Logistic regression assumption

- The target variable $y \in \{0, 1\}$
- Follows a **Bernoulli distribution**
- Model outputs probability:

$$\hat{y}_i = P(y_i = 1 \mid x_i)$$

Likelihood function (MLE)

For one data point:

$$P(y_i \mid x_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{(1-y_i)}$$

For the whole dataset:

$$L = \prod_{i=1}^m \hat{y}_i^{y_i} (1 - \hat{y}_i)^{(1-y_i)}$$

Log-likelihood (used in practice): MLE maximizes this.

$$\log L = \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Cross-entropy loss (log-loss) is:

$$J = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

This is just the **negative log-likelihood**.

Final equivalence: mathematically equivalent under a Bernoulli distribution.

Maximize *log-likelihood* $\Leftrightarrow$ Minimize *cross-entropy loss*

They differ **only by a minus sign (and scaling)**.

Logistic regression algorithm

The output is the prediction value when the value is closer to 1, the higher the probability that the current sample is classified as class=1, and vice versa.

loss function(cost or objective function)

Where m is the number of data points in the training dataset, y_i is the label of the i - th data point, $\hat{p}_i$ is the prediction value of the data point.

The goal is to estimate the parameters $b_1, \dots, b_n, a$, which maximize the so-called **log likelihood function**

1. **Initialization:** Initialize the coefficients $\mathbf{b}$ and learning rate α .
2. **Calculate Predicted Probabilities:** For each training example $\mathbf{x}_i$, calculate the predicted probability $\hat{p}_i$ using the logistic regression hypothesis:

$$\hat{p}_i = \frac{1}{1 + \exp(-\mathbf{b}^T \mathbf{x}_i)}$$

3. **Compute Cost Function:** Calculate the log-loss (cross-entropy) cost function:

$$J(\mathbf{b}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

where m is the number of training examples and y_i is the actual outcome for example i .

4. **Gradient Descent Update:** Update the coefficients using gradient descent:

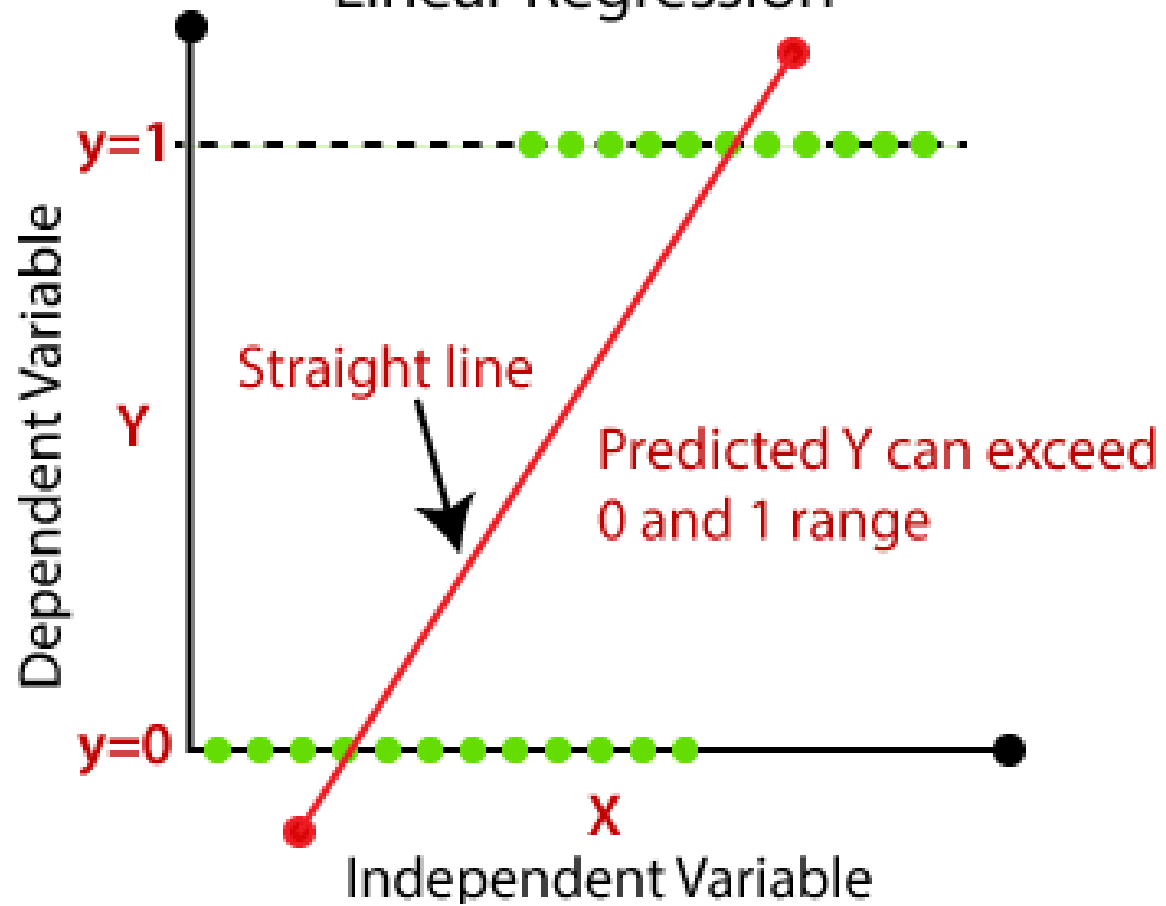
$$b_j := b_j - \alpha \frac{\partial J(\mathbf{b})}{\partial b_j}$$

where $\frac{\partial J(\mathbf{b})}{\partial b_j}$ is the partial derivative of the cost function with respect to coefficient b_j .

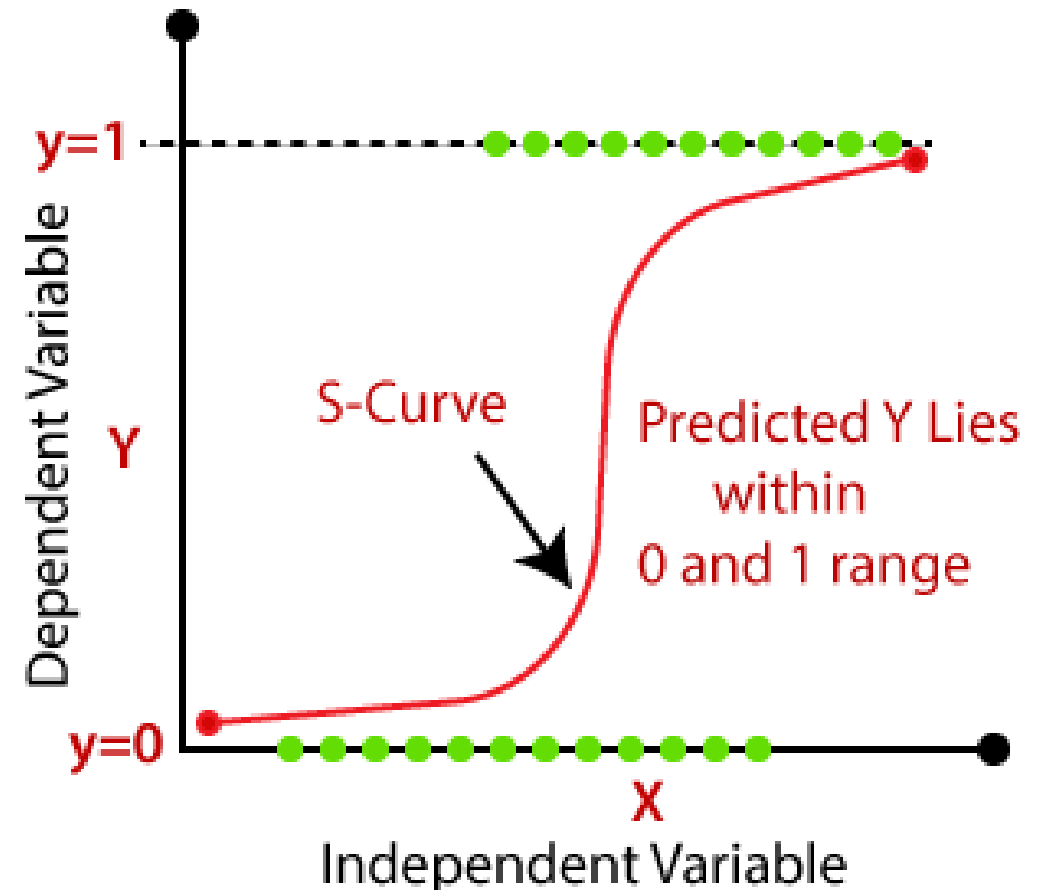
5. **Repeat:** Repeat steps 2-4 until convergence or a maximum number of iterations is reached.
6. **Final Parameters:** After convergence, the optimized coefficients $\mathbf{b}$ are the final parameters of the logistic regression model.
7. **Model Evaluation:** Evaluate the model's performance on a validation or test dataset using appropriate metrics.

8. **Hyper tuning parameters:** experiment with learning rate and others.

Linear Regression



Logistic Regression





Linear Regression	Logistic Regression
Linear regression is used to predict the continuous dependent variable using a given set of independent variables.	Logistic Regression is used to predict the categorical dependent variable using a given set of independent variables.
Linear Regression is used for solving Regression problem.	Logistic regression is used for solving Classification problems.
In Linear regression, we predict the value of continuous variables.	In logistic Regression, we predict the values of categorical variables.
In linear regression, we find the best fit line, by which we can easily predict the output.	In Logistic Regression, we find the S-curve by which we can classify the samples.
Least square estimation method is used for estimation of accuracy.	Maximum likelihood estimation method is used for estimation of accuracy.
The output for Linear Regression must be a continuous value, such as price, age, etc.	The output of Logistic Regression must be a Categorical value such as 0 or 1, Yes or No, etc.
In Linear regression, it is required that relationship between dependent variable and independent variable must be linear.	In Logistic regression, it is not required to have the linear relationship between the dependent and independent variable.
In linear regression, there may be collinearity between the independent variables.	In logistic regression, there should not be collinearity between the independent variable.

K-Nearest Neighbor

K-Nearest Neighbor KNN

- k-Nearest Neighbors is a supervised learning algorithm that classifies new data points based on which training data points they're closest to.

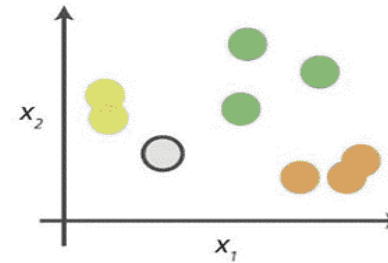
- KNN assumes that closer data points are more like each other.
- The KNN algorithm calculates the probability of the test data belonging to the classes of 'K' training data and class holds the highest probability will be selected.



Can be used for regression ?

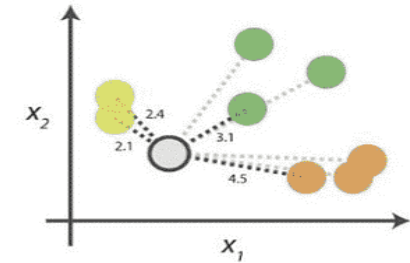
In case of regression, the value is the mean of the 'K' selected training points.

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance			
		2.1	→ 1st NN
		2.4	→ 2nd NN
		3.1	→ 3rd NN
		4.5	→ 4th NN

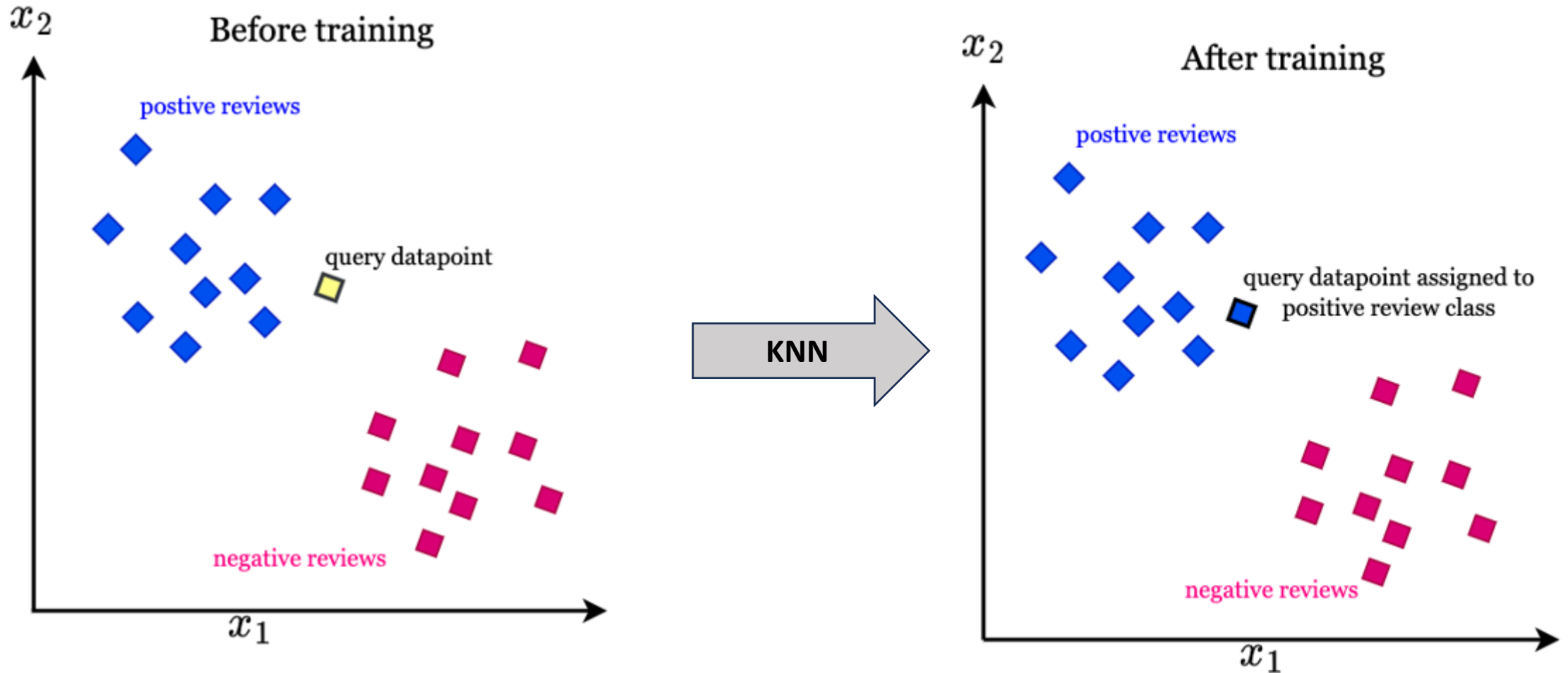
Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

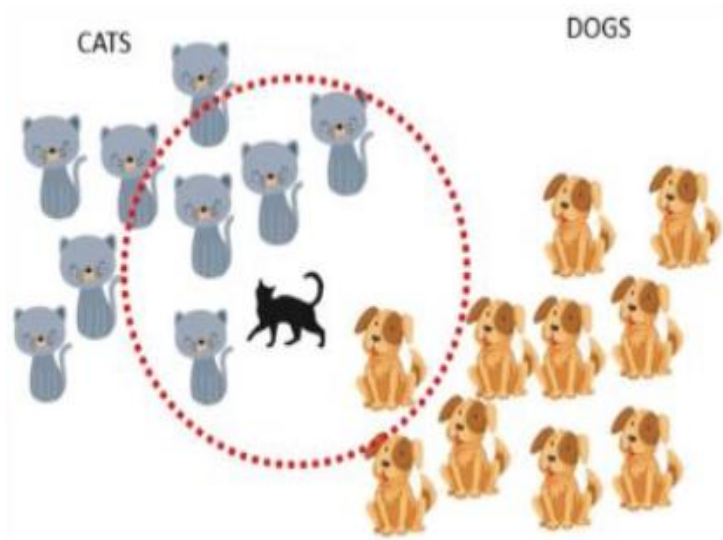
Class	# of votes	
	2	➔ Class wins the vote! Point is therefore predicted to be of class .
	1	
	1	

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

How to categories new data points?

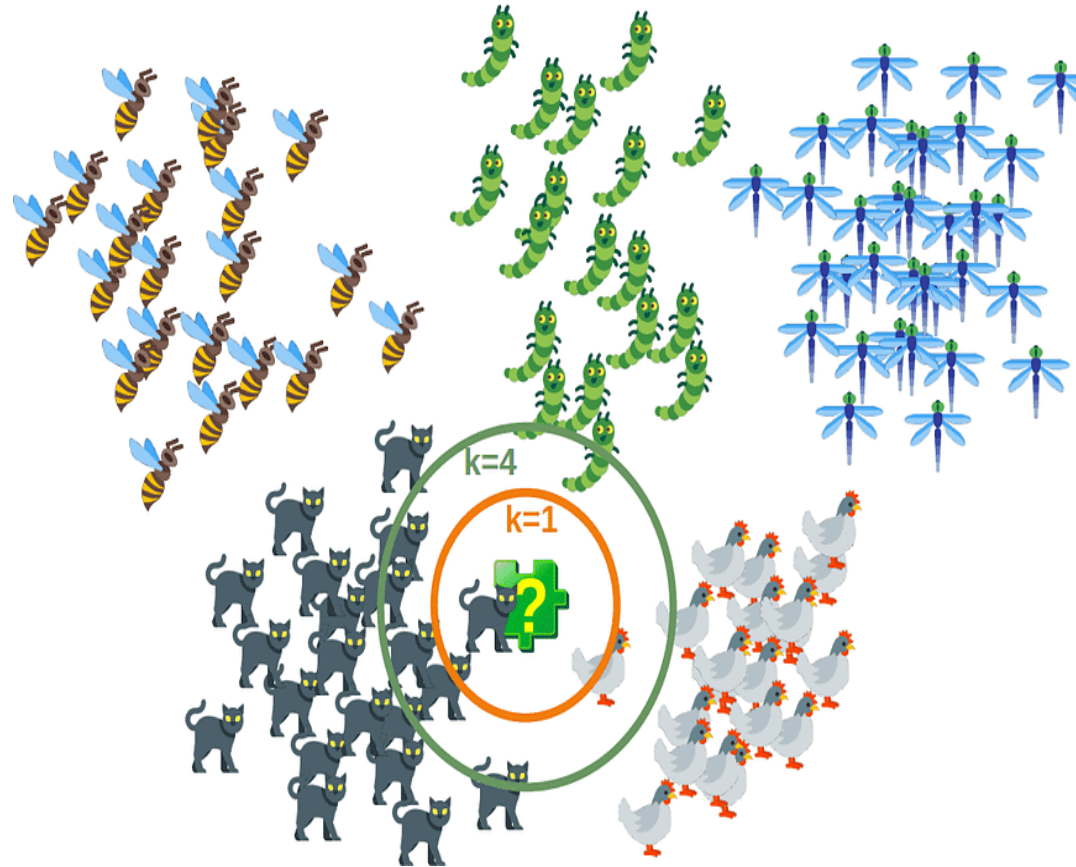


The Binary Classification



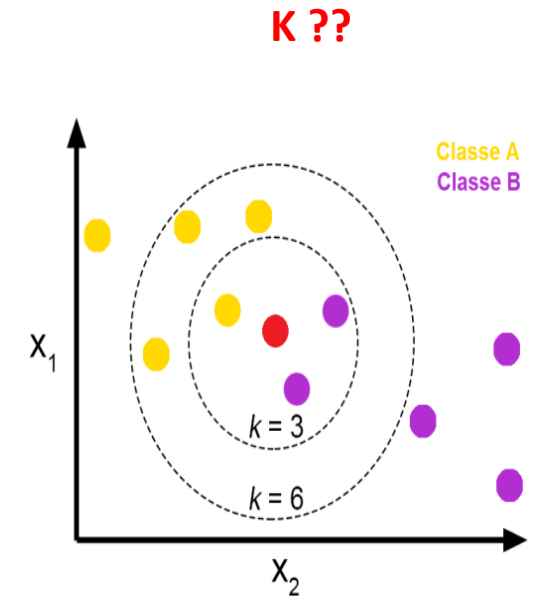
The target variable has two possible outcomes. There are only two possible predicted outcomes in that problem.

Multi-label classification



The prediction for a data instance can be more than 2 possible discrete outcomes.

Example



The number of nearest neighbours to include in the majority of the voting process.

How KNN works?

Step 1: Data Preparation

- Start by gathering and preparing your dataset. Each data point should have features (attributes) that describe it, and a corresponding label (for classification) or target value (for regression).

Step 2: Choosing K

- The parameter 'K' represents the number of nearest neighbors to consider when making predictions. You need to choose a suitable value for K.
- A small K might lead to noisy predictions, while a large K might introduce bias. You can use techniques like cross-validation to determine the best K value for your dataset.

Step 3: Calculating Distance

- To find the K nearest neighbors for a new data point, you need to calculate the distance (e.g., Euclidean distance) between that point and all other points in the dataset. The distance metric depends on the type of data and problem you are working on.

How KNN works?

Step 4: Sorting Neighbors

- Once you have calculated the distances, sort the data points based on their distance from the new data point in ascending order.

Step 5: Selecting K Neighbors

- Choose the K nearest neighbors from the sorted list of distances and associated data points.

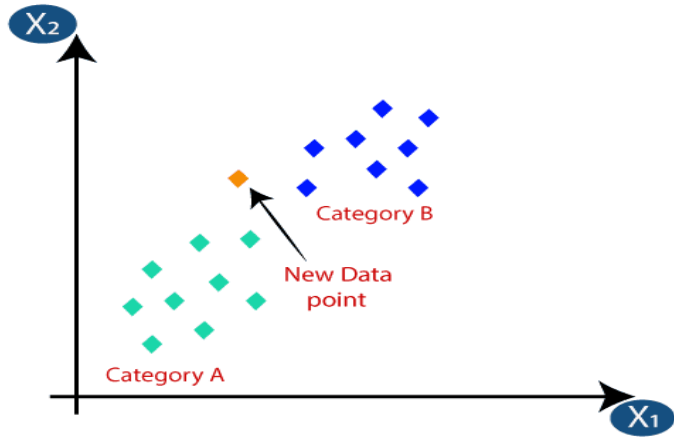
Step 6: Voting (Classification) or Averaging (Regression)

- For classification: Count the occurrences of each class label among the K neighbors and assign the class label with the highest count to the new data point.
- For regression: Calculate the average (or weighted average) of the target values of the K neighbors and assign it as the predicted target value for the new data point.

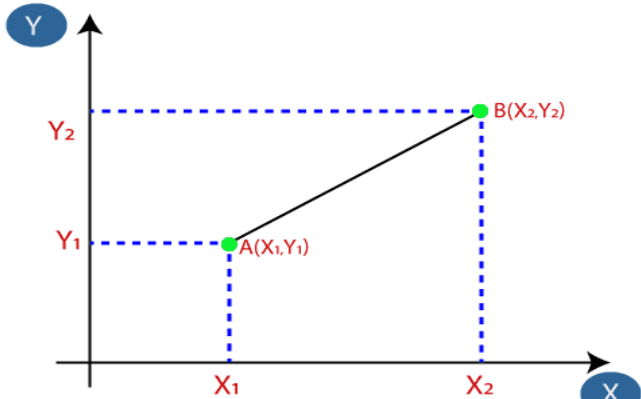
Step 7: Prediction

- After determining the majority class (for classification) or the average target value (for regression), assign that value as the predicted class label or target value for the new data point.

Consider a task of classifying a new data point

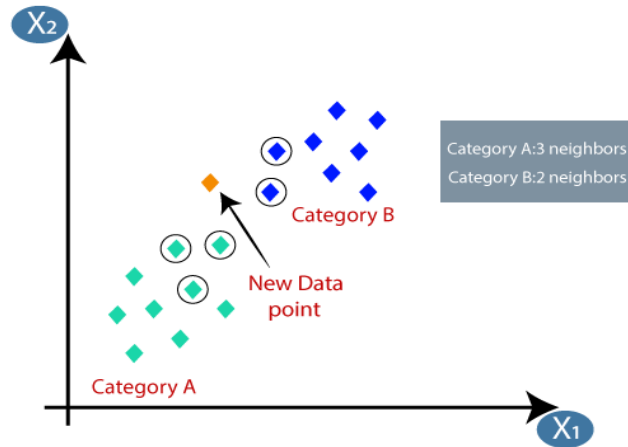


1. Choose the number of neighbors, so we will choose the $k=5$.
2. Calculate the Euclidean distance between the data points.



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

3. Get KNN based on distance.
4. Determine the no. of data points in each category, three nearest neighbors in category A and two nearest neighbors in category B.



5. Based on voting, this new data point must belong to category A. (the one that has maximum neighbors)

Example

The process of choosing the right value of K is referred to as parameter tuning and is of great significance in achieving better accuracy.



Derive a plot between error rate and K denoting values in a defined range. Then choose the K value as having a minimum error rate.

KNN is a Lazy learning algorithm $\rightarrow$ it does **no training**, just stores data.

Distance metrics

Euclidean Distance: Euclidean distance is calculated as the square root of the sum of the squared differences between a new point (x) and an existing point (y).

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Minkowski Distance: For real-valued vector spaces and applied only in a normed vector space, which means in a space where distances can be represented as a vector that has a length and the lengths cannot be negative.

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

Manhattan Distance: This is the distance between real vectors using the sum of their absolute difference.

$$\sum_{i=1}^k |x_i - y_i|$$

Hamming Distance: It is used for categorical variables. If the value (x) and the value (y) are the same, the distance D will be equal to 0. Otherwise, D=1.

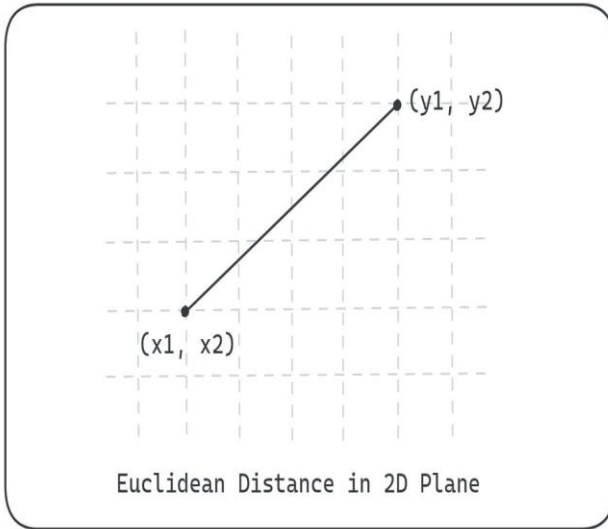
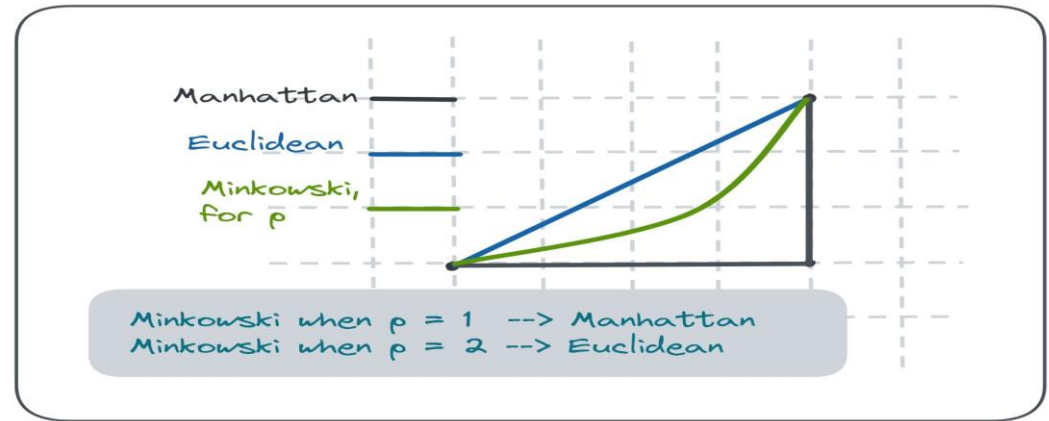
$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

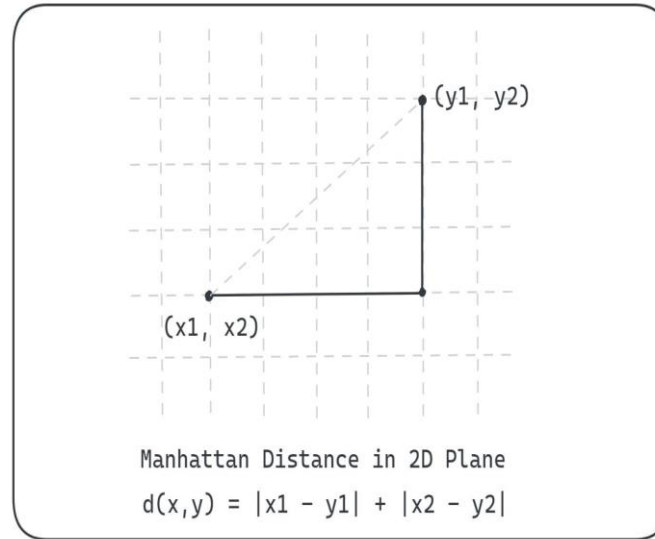
Euclidean, Manhattan & Minkowski

distance



$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



$$d(\mathbf{x}, \mathbf{y}) = |x_1 - y_1| + |x_2 - y_2|$$

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \text{ for } p \geq 1$$

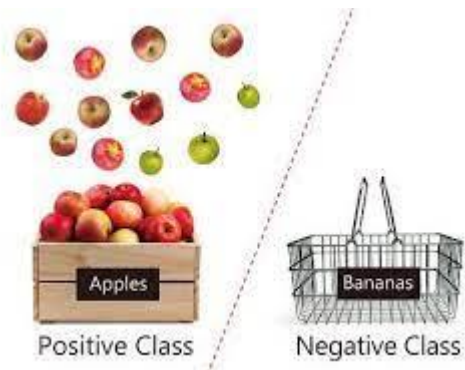
Minkowski distance is a generalization of Euclidean distance

where,

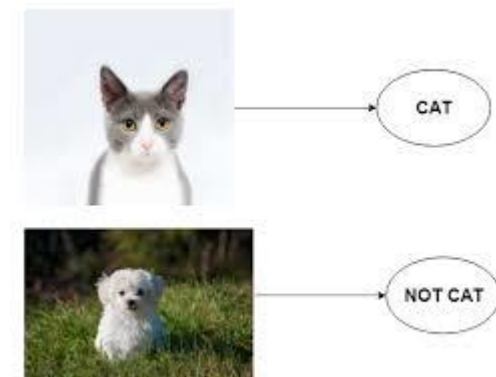
- p is the parameter that determine the type of distance
- n is the number of dimensions (features)

Evaluation criteria of classifier





Tips



What is Positive and Negative?

Positive and negative are nothing but just two classes, for example,

- survived/not survived,
- cancer/not cancer,
- credit card fraud/not fraud,
- spam/not spam, etc.
- It's not only in between two happen/not happen, but it can also be divided into cat/dog, male/female

What is True and False?

When there is a sample data from some population, and we use modeling through which we can predict its class/labels. The **"true"** represents the records that the model was able to identify its class, whereas **"false"** represents the records that the model was not able to identify.



TP / TN / FP / FN

- ❑ **True Positive (TP):** is an outcome where the model **correctly** predicts the **positive** class.
- ❑ **True Negative (TN):** is an outcome where the model **correctly** predicts the **negative** class.
- ❑ **False Positive (FP):** is an outcome where the model **incorrectly** predicts the **positive** class (Type I Error).
- ❑ **False negative** is an outcome where the model **incorrectly** predicts the **negative** class (Type II Error).

<u>True negative</u> Predicted negative Actual negative	<u>False positive</u> Predicted positive Actual negative
<u>False negative</u> Predicted negative Actual positive	<u>True positive</u> Predicted positive Actual positive

Accuracy

It measures how often the classifier correctly predicts. We can define accuracy as the ratio of the number of correct predictions and the total number of predictions.

Accuracy = (Number of correct predictions) / (Overall number of predictions)

$$\textbf{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- Accuracy is useful when the target class is **well balanced** but is not a good choice for the unbalanced classes.

Ex: imagine the scenario where we had 99 images of the dog and only 1 image of a cat present in our training data. Then our model would always predict the dog, and therefore we got 99% accuracy.

Confusion matrix

It is presented as a table in which the predicted class is compared with the actual class.

		PREDICTED VALUES					ACTUAL VALUES	
ACTUAL VALUES		Positive	Negative		PREDICTED VALUES		Positive	Negative
	Positive	True Positive (TP)	False Negative (FN)			Positive	True Positive (TP)	False Positive (FP)
	Negative	False Positive (FP)	True Negative (TN)			Negative	False Negative (FN)	True Negative (TN)

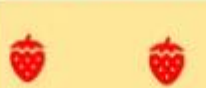
True positive (TP) and True negative (TN) remains in the same place in the matrix placed diagonally to each other.

Confusion matrix

Ex. 1: We had 9 apples and 10 strawberries, but the model identified only 6 apples (true positive) and 8 strawberries (true negative) correctly, moreover, the model predicted 2 strawberries as apple (false positive) and 3 apples (false negative) as strawberries.

		PREDICTED VALUES	
		Positive	Negative
ACTUAL VALUES	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

↔

		PREDICTED VALUES	
		Apple	Strawberry
ACTUAL VALUES	Apple		
	Strawberry		

Confusion matrix

Ex. 2: pregnant/not pregnant are two classes. There are 8 women pregnant and 8 women not pregnant, the model classified 6 pregnant (true positive) and 5 not pregnant (true negative) women correctly, but 3 not pregnant (false positive) women as pregnant and 2 pregnant (false negative) women as not pregnant.

ACTUAL VALUES	PREDICTED VALUES	
	Positive	Negative
	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)



ACTUAL VALUES	PREDICTED VALUES	
	Pregnant	Not Pregnant
	Pregnant	Not Pregnant
Pregnant		
Not Pregnant		

Confusion matrix

Clearer ...

- True Positive (TP): you predicted positive, the real value was positive
- True Negative (TN): you predicted negative, the real value was negative
- False Positive (FP): you predicted positive, the real value was negative
- False Negative (FN): you predicted negative, the real value was positive

True Label Class 1	Correct True Positive for Class 1	Wrong False Positive for Class 2
	Wrong False Positive for Class 1	Correct True Positive for Class 2
True Label Class 2	Predicted Label Class 1	Predicted Label Class 2



		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Confusion matrix

Actual	Positive	Negative
	23	7
	Positive	Negative
	10	60

Predicted



For multiple classes

Actual	Elephant	Monkey	Fish	Lion
	25	3	0	2
	3	53	2	3
	2	1	24	2
	Elephant	Monkey	Fish	Lion
	1	0	2	71

Predicted

Precision & Recall

Precision (also known as Positive Predicted Value): it defined as the number of true positives divided by the sum of true and false positives.

Recall (also known as Sensitivity or True Positive Rate): it defined as the number of true positives divided by the sum of true positives and false negatives.

- **Precision** expresses the proportion of data correctly predicted as positive. In other words, precision helps you measure how often you correctly predicted that a data point belongs to the class your model assigned it to.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** expresses the ability to find all relevant instances in a dataset. Recall measures how good your model is at correctly predicting positive cases.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Ex (spam detection):** If the model's precision is 95%, it means that 95% of the emails classified as spam are actually spam. Meanwhile, a recall of 90% implies that the model correctly identifies 90% of all spam emails.
- **Ex (medical diagnosis):** A model with high precision ensures that the diagnosed cases are highly likely to be accurate, while high recall ensures that the model doesn't miss potentially critical diagnoses.

F1-Score & Specificity

F1 score: It is the weighted average of precision and recall.

Specificity (also known as Selectivity, True Negative Rate): it only the focus is on the negative class. It is the proportion of true negative cases which were correctly identified.

- **F1-Score** is one of the most reliable ways to score how well a classification model performs. A high F1 score is a good indicator that your model performs well, since it achieves high values for both precision and recall.

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall}$$

- **Specificity** is the percentage of all negatives that were correctly predicted as negative. Measures how completely a test or algorithm can identify all possible negatives.

$$Specificity = \frac{TN}{TN + FP}$$

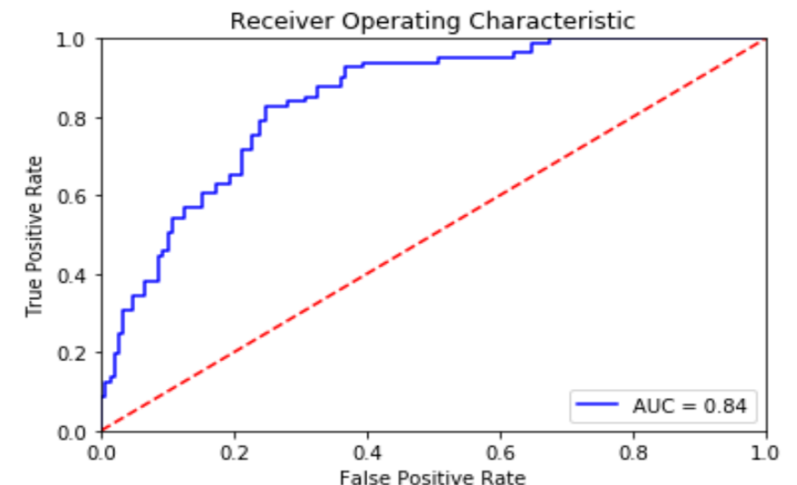
ROC & AUC

The **Receiver Operator Characteristic (ROC)**: is a probability curve that plots the TPR(True Positive Rate) against the FPR(False Positive Rate) at various threshold values and separates the 'signal' from the 'noise'.

The **Area Under the Curve (AUC)** is the measure of the ability of a classifier to distinguish between classes. From the graph, we simply say the area of the curve ABDE and the X and Y-axis.

For simplicity, when AUC is equal to 1, the classifier is able to perfectly distinguish between all Positive and Negative class points. When AUC is equal to 0, the classifier would be predicting all Negatives as Positives and vice versa. When AUC is 0.5, the classifier is not able to distinguish between the Positive and Negative classes.

In a ROC curve, the X-axis value shows False Positive Rate (FPR), and Y-axis shows True Positive Rate (TPR). Higher the value of X means higher the number of False Positives(FP) than True Negatives(TN), while a higher Y-axis value indicates a higher number of TP than FN. So, the choice of the threshold depends on the ability to balance between FP and FN.



Fall-out & Miss rate

Fall-out (False Positive Rate): it determines the probability of determining a positive value when there is no positive value. It is the proportion of actual negative cases that were incorrectly classified as positive.

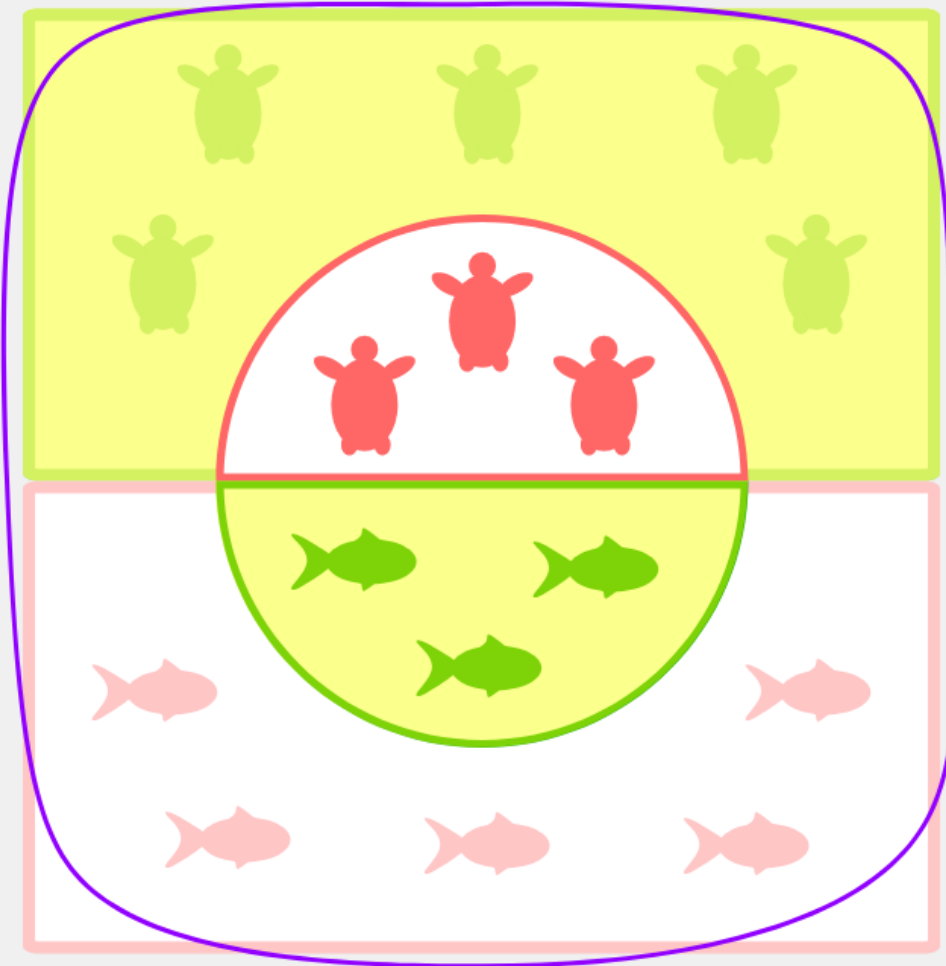
Miss Rate (False Negative Rate): it can be defined as the proportion of positive values that were incorrectly classified as negative examples.

$$\text{Fall Out /False Positive Rate} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{False Negative Rate /Miss Rate} = \frac{\text{FN}}{\text{TP} + \text{FN}}$$

1) Accuracy

For visual learners:



For mathematical learners:

True Negatives + True Positives

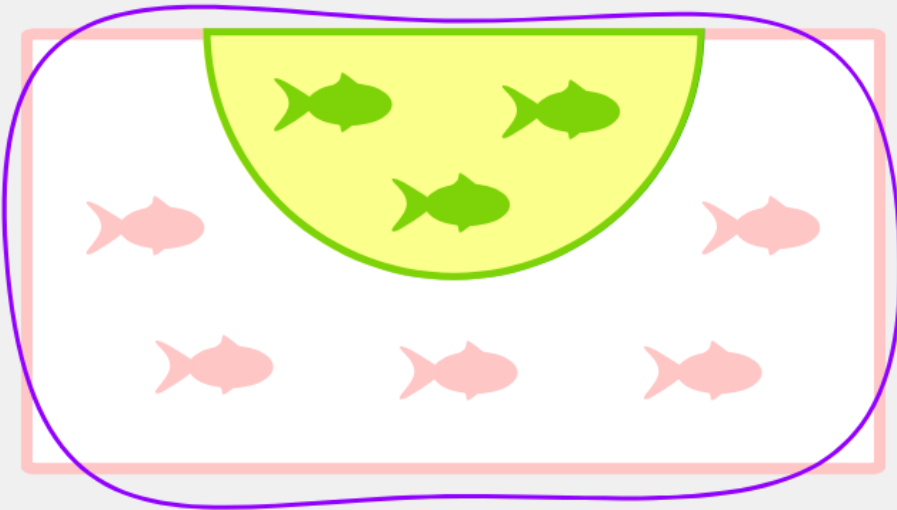
True Negatives + False Positives + True Positives + False Negatives

Everything

- Consider a fishing spot with 1 tuna and 9 sea turtles. We cast our net and pull in a lone sea turtle. Our accuracy is 80%: 8 sea turtles we avoided (True Negatives) + 0 tuna we caught (True Positive), divided by 10 creatures' total.
- 80% is a pretty good grade for a tuna fisherman who didn't catch any tuna! **Data Imbalance**

2) Recall / Sensitivity / True Positive Rate

For visual learners:



For mathematical learners:

$$\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

All Positives

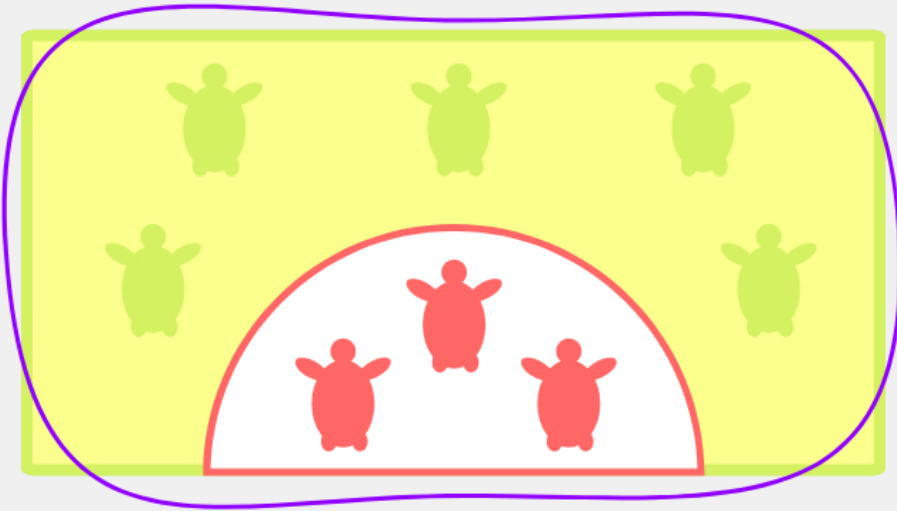


A low recall means the classifier missed many Positives (i.e., incorrectly called them Negatives), which is equivalent to having many False Negatives.

recall measure **completeness**

3) Specificity / True Negative Rate

For visual learners:



For mathematical learners:

$$\frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

All Negatives

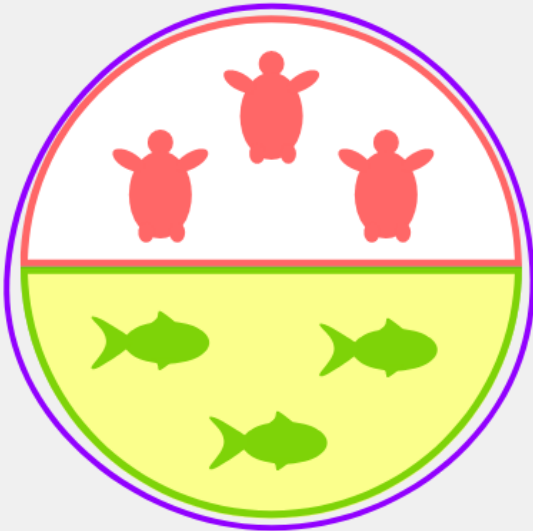


A low specificity means the classifier missed many Negatives (i.e., incorrectly called them Positives), which is equivalent to having many False Positives.

specificity measure **completeness**

4) Precision / Positive Predictive Value

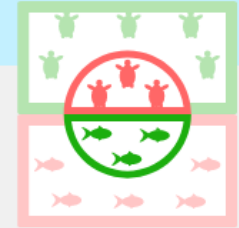
For visual learners:



For mathematical learners:

$$\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Positive Predictions



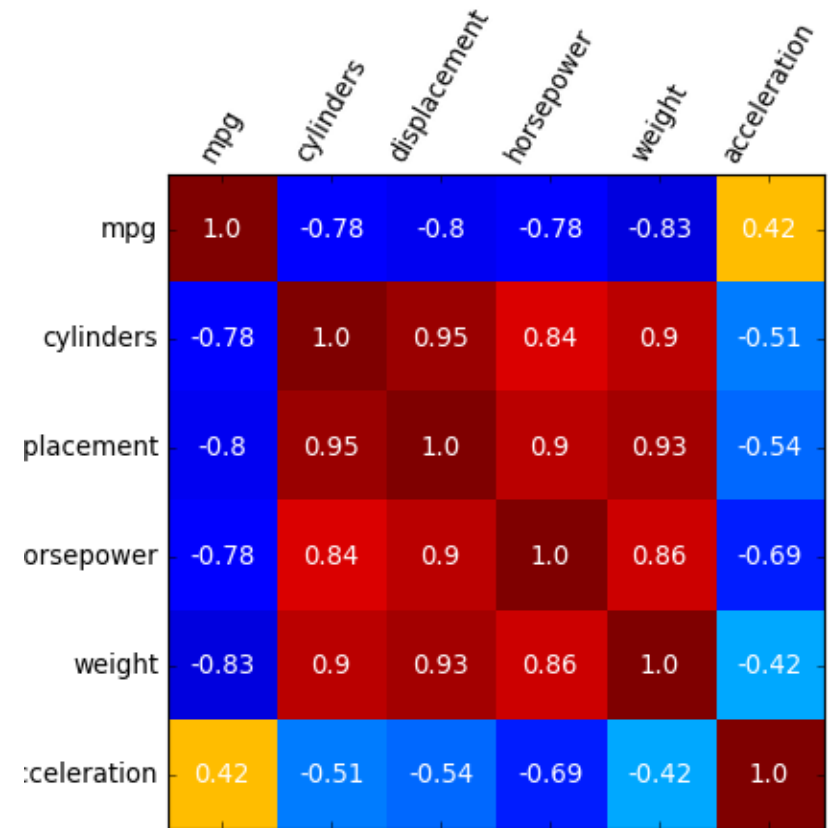
A low precision means many of the predictions of Positive were incorrect (i.e., should have been Negative), which is equivalent to many False Positives.

precision measures **efficiency**

Correlation Matrix

A **correlation matrix** is a statistical technique used to evaluate the relationship between two variables in a data set. The matrix is a table in which every cell contains a correlation coefficient, where 1 is considered a strong relationship between variables (perfect positive), 0 a neutral relationship and -1 a not strong relationship (perfect negative).

- A correlation matrix is typically a square matrix where the diagonal elements are always 1 (since a variable's correlation with itself is always 1), and the other elements represent the correlations between pairs of variables.



Correlation matrix of the Auto-MPG dataset

Advantages of Correlation matrix



Correlation matrices are useful for understanding the relationships between multiple variables in a dataset.



They can help in identifying patterns, dependencies, and potentially redundant features.



Correlation matrices are commonly used in exploratory data analysis, feature selection, and in some cases, preprocessing data for machine learning algorithms.

Example

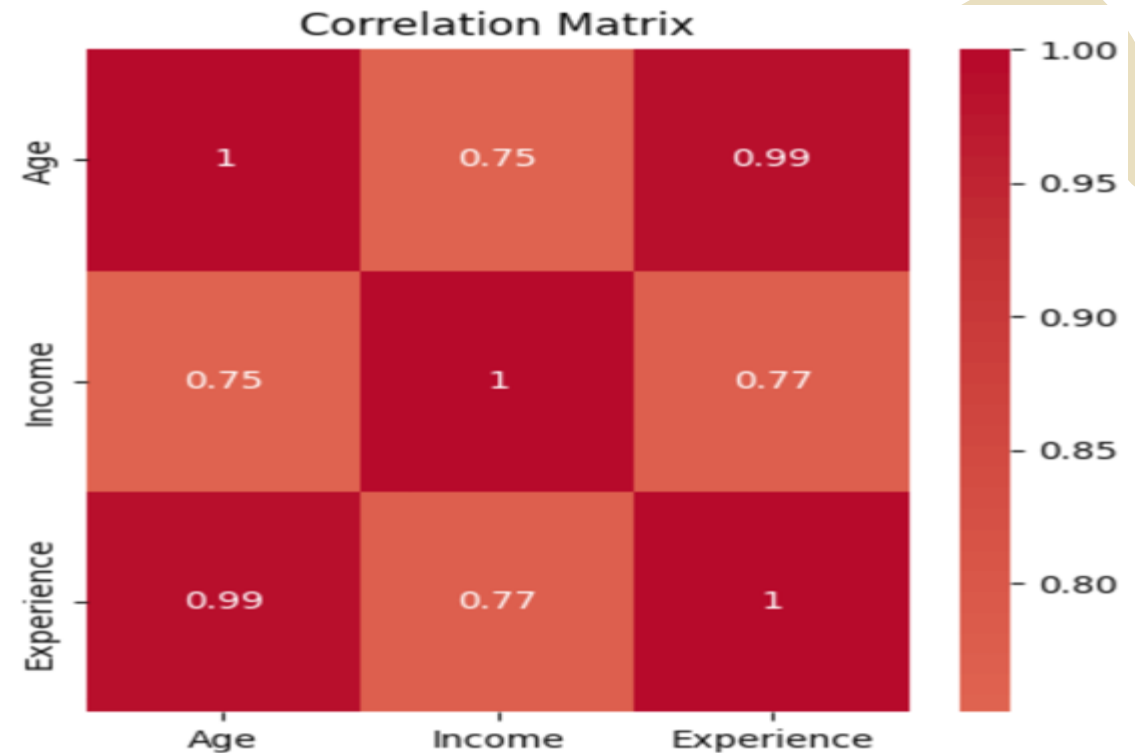
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Example dataset
data = {
    'Age': [25, 30, 40, 35, 28],
    'Income': [50000, 60000, 75000, 55000, 62000],
    'Experience': [3, 5, 12, 8, 4]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Calculate the correlation matrix
correlation_matrix = df.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()
```



- `annot=True`, it adds the actual correlation coefficient values to the cells of the heatmap.
- Each cell of the heatmap corresponds to a pair of variables, and the number displayed in the cell is the correlation coefficient between those variables.

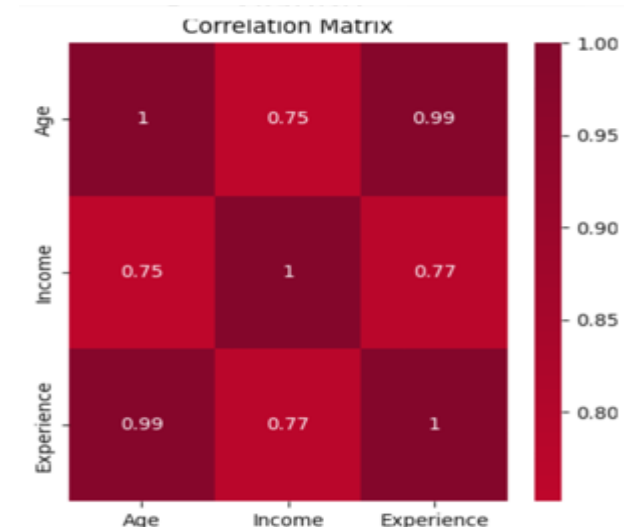
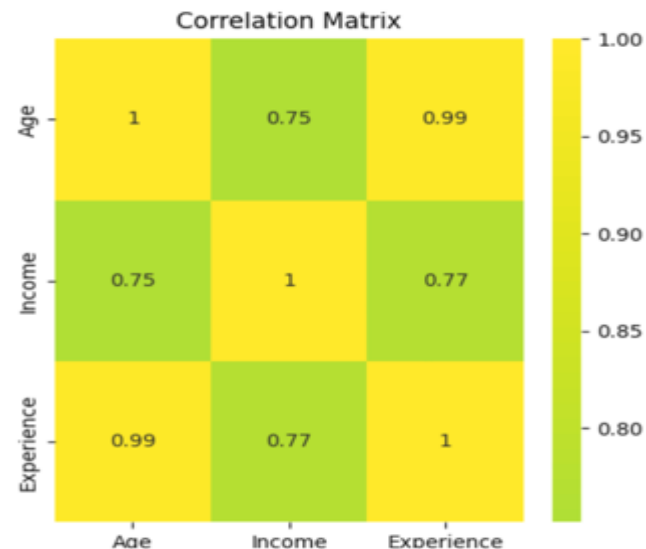
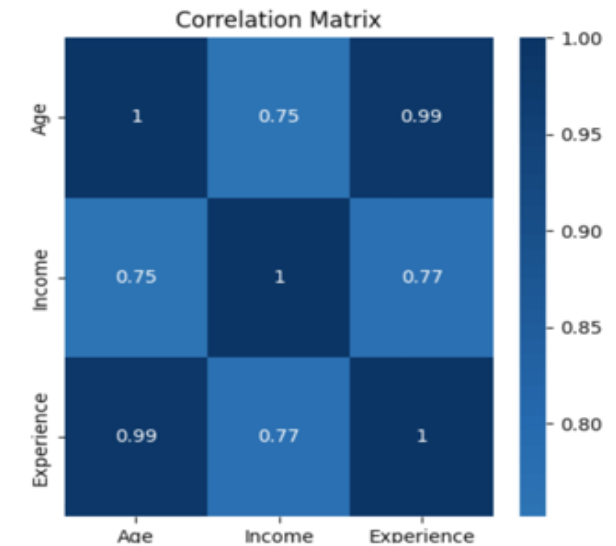
Example

```
# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()
```

- In the context of creating visualizations, particularly heatmaps, a "cmap" refers to the color map or color palette used to represent different values in the data using colors.
- Color maps define how data values are mapped to colors in the visualization. Different color maps can be chosen based on the data and the desired visual effect.

- Here are some commonly used color maps (cmaps) available in Python libraries like Matplotlib and Seaborn:

- 'viridis'
- 'plasma'
- 'cividis'
- 'inferno'
- 'coolwarm'
- 'RdBu'
- 'YlGnBu'
- 'YlOrRd'



Test Your Knowledge

MCQ

Logistic Regression is mainly used for:

- A) Predicting continuous values
- B) **Predicting probabilities of discrete classes**
- C) Feature scaling
- D) Reducing dimensionality

Which metric should be prioritized for **imbalanced** datasets?

- A) Accuracy
- B) **Recall**
- C) R-squared
- D) RMSE

T / F

- Logistic Regression can output probabilities between 0 and 1

T

- “MLE is always equivalent to cross-entropy”

F Only true when the assumed distribution matches the loss (e.g., Bernoulli → cross-entropy)

- Increasing K in KNN always reduces bias

F

THANK
you